

BACKEND ENGINEERING PROJECT

FinTrack

Setup & Verification Guide

from `git clone` to proving every guarantee

Node.js

Express

PostgreSQL

MongoDB

Redis

BullMQ

Prisma

Docker

Nginx

k6

Every command in this document was executed against the codebase.
Outputs shown are real, not illustrative.

github.com/kanhaiyak23/FinTrack · 153 tests passing · 19 recorded decisions

Contents

Every entry is clickable. Each section has a “back to contents” link in its corner.

- 1 Do I still need Docker for local development?

- 2 What the system is, in one picture

- 3 From clone to running — the exact process

- 4 The two ways to run it

- 5 Feature walkthrough with real output

- 6 The six guarantees, and how to prove each

- 7 Looking inside the three databases

- 8 Tests, benchmarks and load results

- 9 Scaling it and proving it scaled

- 10 Troubleshooting

- 11 The 90-second demo

How to read this

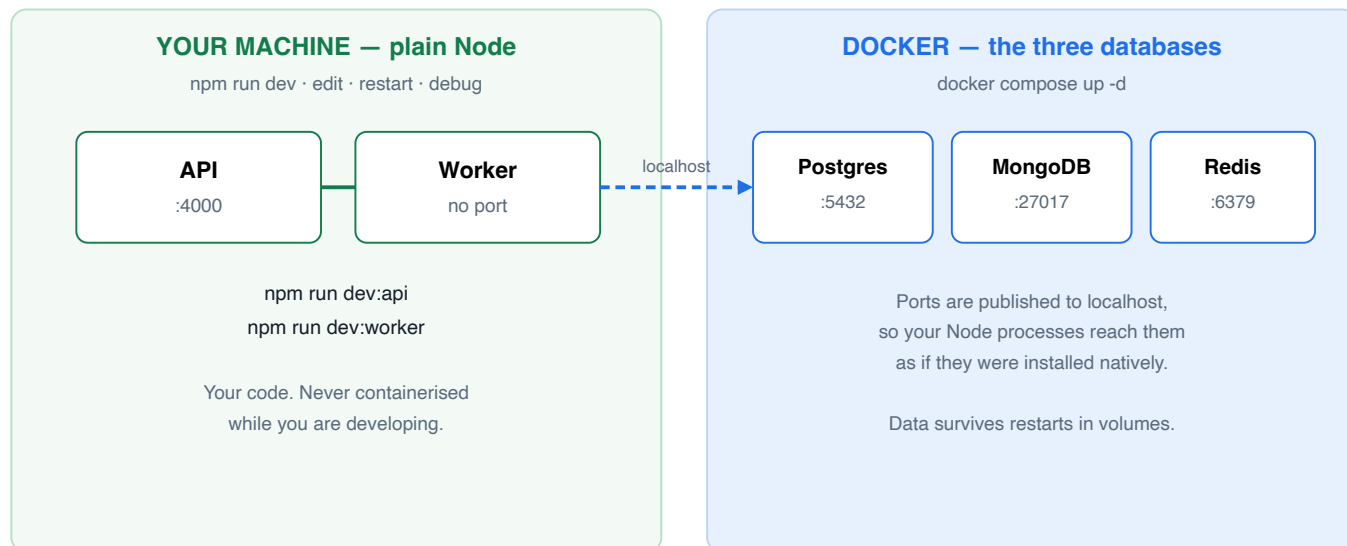
Marker	Meaning
COMMAND	Type this. Blue left border.
OUTPUT	What you should see. Green left border.
WRONG	A failure mode, and what it means.
WATCH OUT	Something that will bite you if skipped.

1. Do I still need Docker for local development?

[contents ↑](#)

Yes — but only for the three databases, never for your own code. This is the single most useful thing to understand about running this project.

FinTrack needs PostgreSQL, MongoDB and Redis. You have two honest options: install all three natively on your Mac and manage three services, or run one command and let Docker provide them. The project assumes the second. **Your API and worker still run as ordinary Node processes** with `npm run dev` — you edit a file, it restarts, you set breakpoints, exactly as normal.



Local development: your code runs on the host, only the databases are containerised.

So what does Docker actually give you here?

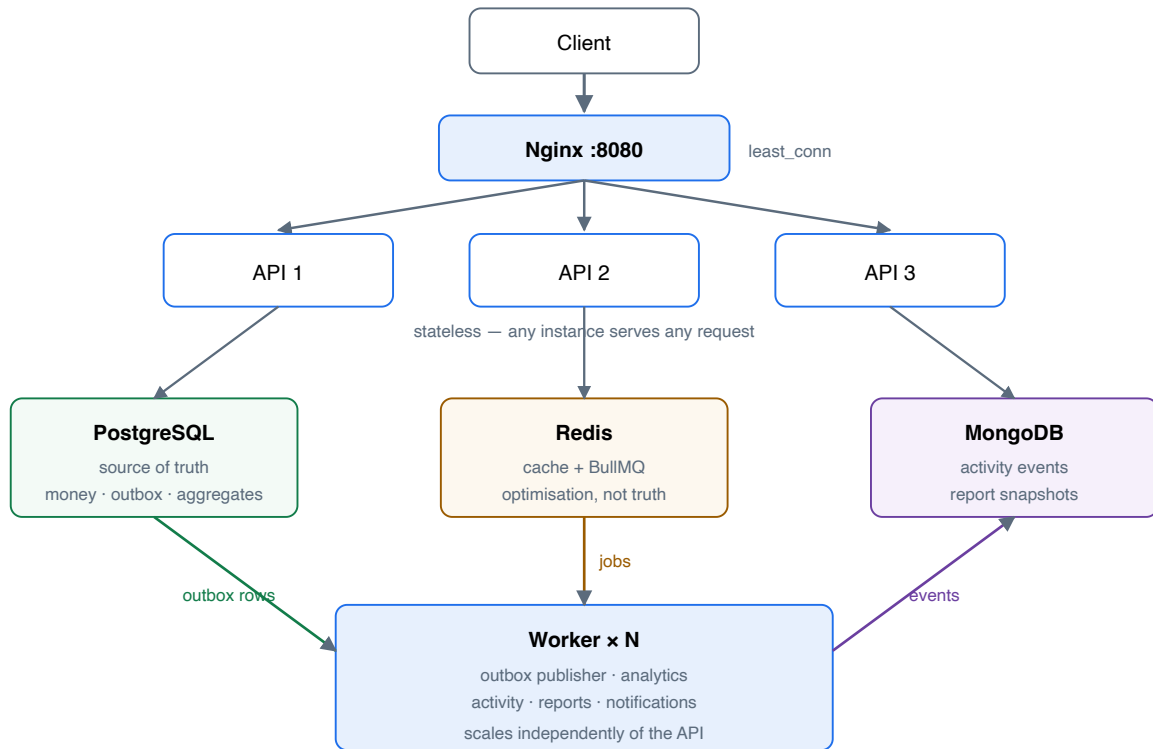
Without Docker	With Docker
Install Postgres 16, MongoDB 8 and Redis 7 by hand; keep three brew services running	One command, correct versions, isolated from anything else on your Mac
Version drift between your machine and anyone else's	Everyone gets the same versions, pinned in <code>docker-compose.yml</code>
Cleaning up means uninstalling services	<code>docker compose down -v</code> and it is gone
Nowhere to demonstrate Nginx or multiple API instances	<code>--profile app --scale api=3</code> and it is real

The short version. `docker compose up -d` once per session to get your databases. Then `npm run dev:api` and `npm run dev:worker` like any other Node project. You only containerise your *own* code when you want to demo the full scaled stack behind Nginx — see [section 4](#).

Both processes are required. The API alone will accept writes and return `201`, but analytics, activity history and reports stay empty, because a worker is what updates them. If your portfolio looks empty, this is almost always why.

2. What the system is, in one picture

[contents](#) ↑

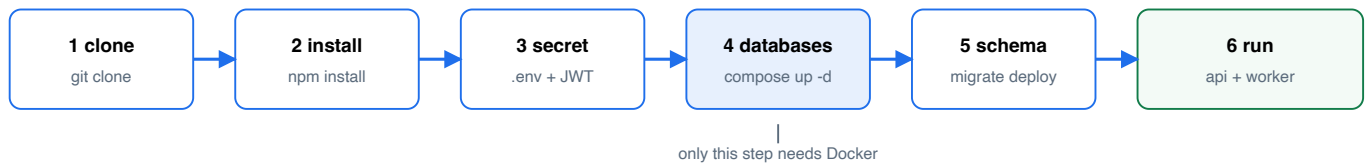


Two process types, not six services. The API answers requests; the worker does everything expensive.

Piece	Why it is here
PostgreSQL	Money needs ACID transactions, foreign keys and CHECK constraints. It is the only thing allowed to be authoritative.
Redis	Caches expensive analytics and backs the job queue. If it dies, the system gets slower, not wrong.
MongoDB	Activity events genuinely differ in shape — a trade has symbol/quantity/price, a login does not. A relational table would be mostly NULL.
BullMQ	Retries, exponential backoff and a visible failed set, none of which you want to hand-roll.
Nginx	Makes “the API is stateless” a testable claim rather than an assertion.

3. From clone to running — the exact process

[contents](#) ↑



Six steps. Steps 1–3 and 5–6 are ordinary Node work; only step 4 involves Docker.

Step 0 — prerequisites

Need	Check	Install
Node 20+	<code>node -v</code>	<code>brew install node</code>
Container runtime	<code>docker info</code>	<code>brew install colima docker docker-compose</code>
<code>jq</code> (used in every example)	<code>jq --version</code>	<code>brew install jq</code>
k6 (only for load tests)	<code>k6 version</code>	<code>brew install k6</code>

Colima is the container runtime this project was built against — it is open source, has no GUI and no licensing questions. Start the VM once:

```
colima start --cpu 4 --memory 4 --disk 40
```

One-time fix most people hit. Homebrew installs `docker-compose` as a standalone binary, so `docker compose` (with a space) may report *unknown command*. Link it as a CLI plugin:

```
mkdir -p ~/.docker/cli-plugins
ln -sf $(brew --prefix)/opt/docker-compose/bin/docker-compose ~/.docker/cli-plugins/docker-compose
docker compose version
```

Steps 1–3 — clone, install, configure

```
git clone https://github.com/kanhaiyak23/FinTrack.git
cd FinTrack
npm install
```

npm 11 blocks install scripts by default, and Prisma needs its postinstall to fetch the query engine. If you see an `allow-scripts` warning:

```
npm approve-scripts prisma @prisma/engines @prisma/client
npm install
```

```
cp .env.example .env
openssl rand -hex 32          # paste the result into JWT_SECRET= inside .env
```

The app **refuses to boot** without a secret of at least 32 characters. That is deliberate: a process running with a guessable signing key is worse than one that will not start. You can see the refusal for yourself:

```
Invalid environment configuration:
JWT_SECRET: JWT_SECRET must be at least 32 characters
```

Steps 4–5 — databases and schema

```
docker compose up -d
docker compose ps          # wait until all three report (healthy)
npx prisma migrate deploy  # tables, indexes, constraints
npx prisma generate        # the typed client
```

SERVICE	STATUS
mongodb	Up 5 seconds (healthy)
postgres	Up 5 seconds (healthy)
redis	Up 5 seconds (healthy)

Step 6 — run it

Two terminals. Both are needed.

```
# terminal 1
npm run dev:api          # http://localhost:4000

# terminal 2
npm run dev:worker       # no port — this is what updates analytics
```

```
[13:12:31] INFO: mongodb connected
[13:12:31] INFO: fintrack api listening
port: 4000 env: "development"
```

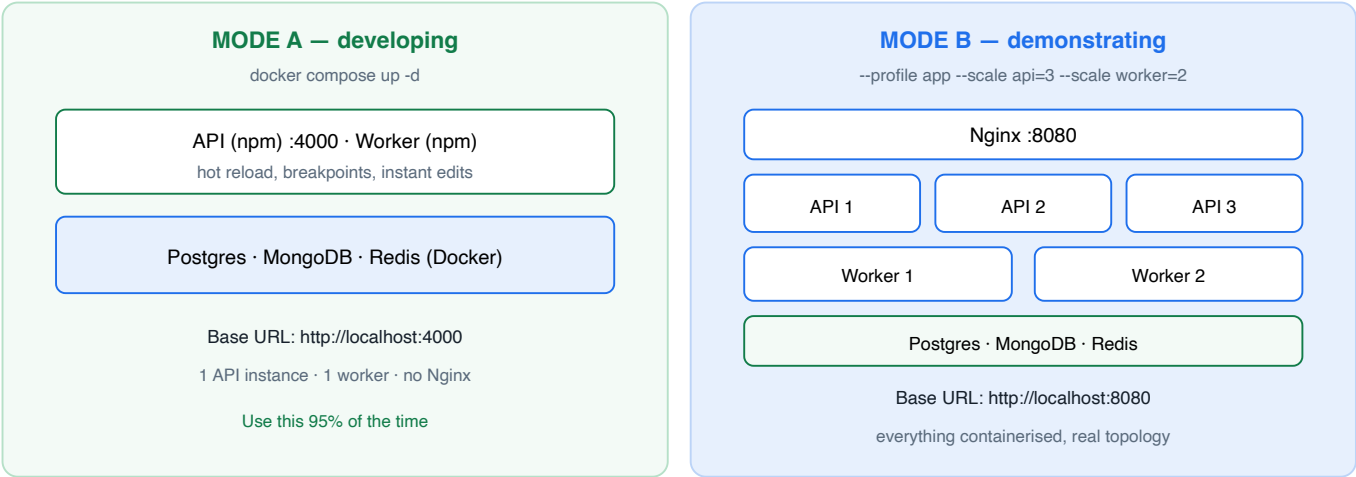
Confirm everything is talking to everything:

```
curl -s http://localhost:4000/health | jq
```

```
{
  "status": "ok",
  "instance": "api-local",
  "uptimeSeconds": 1,
  "checks": {
    "postgres": { "status": "up", "latencyMs": 1.28 },
    "mongodb": { "status": "up", "latencyMs": 1.24 },
    "redis": { "status": "up", "latencyMs": 0.89 }
  }
}
```

4. The two ways to run it

contents ↑



Mode A for writing code. Mode B for showing someone that it scales.

	Mode A — local	Mode B — full stack
Command	npm run dev:api + dev:worker	docker compose --profile app up -d
Base URL	localhost:4000	localhost:8080
Code changes	instant reload	needs an image rebuild
Instances	1 API, 1 worker	3 API, 2 workers, Nginx
Good for	writing and debugging	demoing, load testing, scaling proof

Never run both at once, and stop Mode B before running tests. They share one Postgres and one Redis, so a containerised worker will silently consume the jobs your local worker or your test suite is waiting on. This cost me real debugging time — five queue tests failed with no apparent cause.

```
docker compose --profile app stop api worker nginx
```


Trade amounts come back as `1000.0000` and `2000.0000` — **computed server-side** as quantity × price. Sending your own `amount` on a trade is rejected, or a client could set its own price.

Money is always a string, never a JSON number:

```
curl -s $API/accounts -H "Authorization: Bearer $TOKEN" | jq '.accounts[0].balance'
```

```
"98250.0000"    ← quoted
```

JSON numbers are IEEE-754 doubles. A client parsing `12345678.91` can silently receive something else, so every monetary value crosses the wire as a decimal string.

Overdraft refused:

```
post '{"type":"WITHDRAWAL","accountId":"'$ACC',"amount":"999999"}' | jq
```

```
{
  "error": {
    "message": "Insufficient funds",
    "details": ["available 98250.0000, required 999999.0000"],
    "requestId": "54fb62de-..."
  }
}
```

Short selling refused:

```
"Insufficient holdings"
"TCS: holding 0.00000000, attempted to sell 1.00000000"
```

A deposit carrying trade fields is refused, not silently ignored — an unexpected field on a money movement means the client believes it is doing something the server is not.

5.3 Analytics

Wait two seconds after writing; a worker updates these, not the request.

```
curl -s $API/analytics/portfolio -H "Authorization: Bearer $TOKEN" | jq
```

```
{
  "cash": { "total": "98750.0000", "accounts": 1 },
  "holdings": [{
    "symbol": "INFX",
    "quantity": "15.00000000",
    "averageBuyPrice": "150.0000",
    "costBasis": "2250.0000",
    "realizedPnl": "500.0000"
  }],
  "totals": { "symbols": 1, "costBasis": "2250.0000",
    "marketValue": null, "unrealizedPnl": null }
}
```

Check the arithmetic yourself — this is the number to be able to defend.

Bought 10 @ 100 and 10 @ 200 → 20 units costing 3,000 → average **150**.

Sold 5 @ 250 → realised $(250 - 150) \times 5 = \mathbf{500}$.

Left with 15 units at a basis of $3,000 - (5 \times 150) = \mathbf{2,250}$.

`marketValue` and `unrealizedPnl` are `null` on purpose. There is no price feed, and a number derived from cost basis would look like a valuation without being one. The P&L endpoint states `"method": "WEIGHTED_AVERAGE"` so a consumer never has to guess which cost-basis rule produced the figure.

5.4 Activity history — and why MongoDB is here

```
curl -s "$API/activity?limit=10" -H "Authorization: Bearer $TOKEN" \
  | jq -r '.events[] | "{(.eventType) | (.entityType)}"
```

```
TRADE_EXECUTED      transaction
TRADE_EXECUTED      transaction
DEPOSIT_COMPLETED   transaction
LOGIN               user
USER_REGISTERED     user
```

Now look at the `metadata` of each. A trade carries `symbol`, `quantity`, `price`. A registration carries an `email`. A plan carries a `frequency`. One relational table holding all of those would be mostly NULL columns — **that is the concrete answer to “why not just use Postgres JSONB?”**: this collection needs no joins and no transactions, so it gains nothing from being relational.

5.5 Reports — the async boundary made visible

```
TODAY=$(date +%F)
curl -s -i "$API/reports/daily?date=$TODAY" -H "Authorization: Bearer $TOKEN" \
  | grep -iE '^HTTP|retry-after'
```

```
HTTP/1.1 202 Accepted
retry-after: 10
```

202, not 404. The report does not exist yet — and asking for it queued its generation. Ask again a few seconds later:

```
{
  "transactionCount": 7,
  "tradingVolume": "4250.0000",
  "deposits": "100600.0000",
  "withdrawals": "60.0000",
  "buyValue": "3000.0000",
  "sellValue": "1250.0000",
  "realizedPnl": "500.0000",
  "activeInvestmentPlans": 0,
  "activityEvents": 8,
  "activeDays": 1,
  "openPositions": 1,
  "timezone": "Asia/Kolkata"
}
```

Your figures will differ — they reflect what you posted. What matters is that they come from pre-aggregated tables rather than a scan, and that `timezone` states which day boundary was used.

6. The six guarantees, and how to prove each

[contents ↑](#)

These are the properties worth demonstrating in an interview. Each one is *observable*, not asserted.

#	Guarantee	How you see it
6.1	A retried POST creates one transaction	<code>201</code> then <code>200</code> ; five concurrent retries return identical ids
6.2	Concurrent withdrawals cannot both pass	two × 60 against 100 → one <code>201</code> , one <code>422</code> , balance <code>40.0000</code>
6.3	Redis failing costs latency, not correctness	stop Redis → still <code>200</code> , same figures, <code>x-cache: MISS</code>
6.4	An event is never lost between commit and queue	stop worker → row waits in the outbox → restart → drains to 0
6.5	The same event twice changes nothing twice	<code>processed_events</code> ledger / Mongo unique index
6.6	Aggregates are order-independent	three trades fired concurrently still give 2250 / 500

6.1 Idempotency

```
KEY="demo-$(date +%s)"
curl -s -o /dev/null -w 'first: %{http_code}\n' -X POST $API/transactions \
  -H "Authorization: Bearer $TOKEN" -H 'Content-Type: application/json' \
  -H "Idempotency-Key: $KEY" -d '{"type":"DEPOSIT","accountId":"'ACC',"amount":"500"}'
curl -s -o /dev/null -w 'second: %{http_code}\n' -X POST $API/transactions \
  -H "Authorization: Bearer $TOKEN" -H 'Content-Type: application/json' \
  -H "Idempotency-Key: $KEY" -d '{"type":"DEPOSIT","accountId":"'ACC',"amount":"500"}'
```

```
first: 201
second: 200
```

200, not 201. A `201` would claim a second transaction was created — exactly what idempotency prevented. The body is the original transaction with `"replayed": true`.

Now under real concurrency — five simultaneous retries of one key:

```
KEY="race-$(date +%s)"
for i in 1 2 3 4 5; do
  curl -s -X POST $API/transactions -H "Authorization: Bearer $TOKEN" \
    -H 'Content-Type: application/json' -H "Idempotency-Key: $KEY" \
    -d '{"type":"DEPOSIT","accountId":"'ACC',"amount":"250"}' | jq -r .transaction.id &
done; wait
```

Five identical ids. One transaction, one balance movement. Two mechanisms are at work: a lookup inside the row lock catches sequential retries, and a unique index catches the concurrent ones — one request loses that race and is resolved to the winner's row.

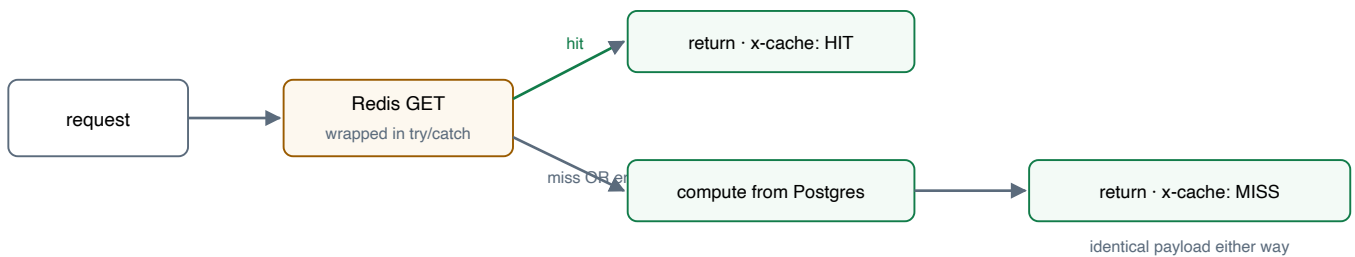
6.2 Lost updates

```
for i in 1 2; do
  curl -s -o /dev/null -w "%{http_code}\n" -X POST $API/transactions \
    -H "Authorization: Bearer $TOKEN" -H 'Content-Type: application/json' \
    -d '{"type":"WITHDRAWAL","accountId":"'ACC2',"amount":"60"}' &
done; wait
```

```
201
422
final balance: 40.0000
```

Without `SELECT ... FOR UPDATE` both requests read a balance of 100, both conclude 60 is affordable, and the account lands at -20. The lock serialises them; the `CHECK (balance >= 0)` constraint is the database-level backstop if the service is ever wrong.

6.3 Cache failure



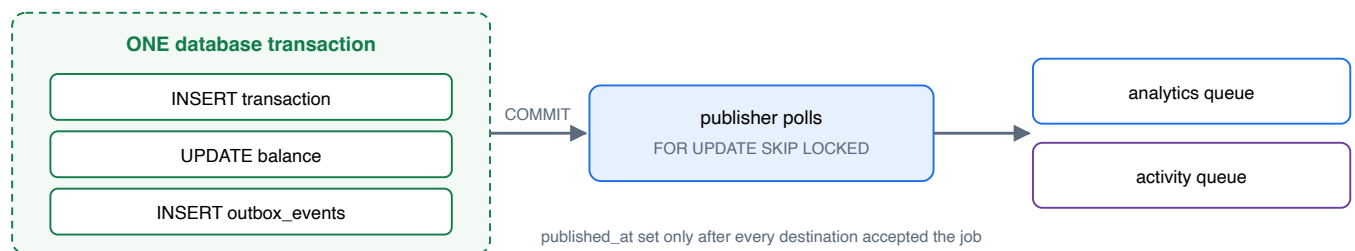
A cache miss and a cache outage are the same thing from the caller's point of view.

```
docker compose stop redis
curl -s -o /dev/null -w 'status: %{http_code}\n' $API/analytics/portfolio -H "Authorization: Bearer $TOKEN"
curl -s $API/health | jq '.status, .checks.redis.status'
docker compose start redis
```

```
status: 200
"ok"
"down"
```

Still `200`, same correct figures, and `/health` reports healthy overall with Redis marked down. Postgres down is the opposite — `503` — because nothing can be served correctly without the source of truth.

6.4 The outbox



The event and the money move in the same commit. Nothing can succeed while the other fails.

Stop the worker, write a transaction, and watch the event wait durably:

```
docker compose exec -T postgres psql -U fintrack -d fintrack -c \
  "SELECT event_type, published_at FROM outbox_events WHERE published_at IS NULL;"
```

The row sits there with `published_at` `NULL`. Start the worker and it drains to zero. The event and the balance change were written in one transaction, so there is no window in which the money moved but the event vanished.

6.6 Order independence — a bug this document found

This was a real defect, discovered while writing this guide. Weighted-average cost basis is order-dependent, and the worker processes jobs concurrently. Three trades on one account were claimed within milliseconds and then raced for the holding row lock — which serialises the writes but does *not* order them. The result: a cost basis of **2,500** and realised P&L of **750**, instead of 2,250 and 500. Exactly what you get by applying the sale before the second purchase. Nothing errored; the numbers were simply plausible and wrong.

Fix (ADR-019): a trade event no longer mutates the holding. It recomputes it by replaying that symbol's trades from the transaction log, so the result is identical whatever order events arrive in — and a late event now *heals* the aggregate instead of corrupting it.

```
for t in BUY_100 BUY_200 SELL_250; do ... done      # fire all three back to back
sleep 4
curl -s $API/analytics/portfolio -H "Authorization: Bearer $TOKEN" \
  | jq '.holdings[] | select(.symbol=="WIPRO")'
```

```
{
  "quantity": "15.00000000",
  "averageBuyPrice": "150.0000",
  "costBasis": "2250.0000",
  "realizedPnl": "500.0000"
}
```

Must be these four values **every time**. If you ever see `2500` and `750`, the sale was applied before the second purchase.

7. Looking inside the three databases

[contents ↑](#)

PostgreSQL

```
docker compose exec -it postgres psql -U fintrack -d fintrack
```

```
\dt          -- tables
\di          -- indexes
\d transactions -- one table in full, constraints included
```

The business rules the database enforces itself — not the application:

```
SELECT conrelid::regclass AS "table", conname
FROM pg_constraint WHERE contype='c' AND connamespace='public'::regnamespace ORDER BY 1;
```

accounts		chk_accounts_balance_non_negative
investment_plans		chk_plans_amount_positive
subscriptions		chk_subscriptions_ended_at
transactions		chk_transactions_amount_positive
transactions		chk_transactions_price_positive
transactions		chk_transactions_quantity_positive
transactions		chk_transactions_trade_fields

Try to break one by hand. It should refuse — this is the demonstration that correctness does not depend on the application being right:

```
UPDATE accounts SET balance = -1 WHERE id = (SELECT id FROM accounts LIMIT 1);
```

```
ERROR:  new row violates check constraint "chk_accounts_balance_non_negative"
```

```
INSERT INTO transactions (id, account_id, type, amount)
VALUES (gen_random_uuid(), (SELECT id FROM accounts LIMIT 1), 'BUY', 100);
```

```
ERROR:  new row violates check constraint "chk_transactions_trade_fields"
-- a BUY with no symbol
```

Operational queries you will actually use:

```
-- is anything stuck in the outbox?
SELECT event_type, count(*), count(*) FILTER (WHERE published_at IS NULL) AS unpublished
FROM outbox_events GROUP BY 1;

-- rows that keep failing to publish
SELECT id, event_type, attempts, last_error FROM outbox_events
WHERE published_at IS NULL AND attempts > 0;

-- the pre-aggregated state the analytics endpoints read
SELECT * FROM portfolio_holdings;
SELECT * FROM daily_user_aggregates ORDER BY day DESC LIMIT 5;
```

MongoDB

```
docker compose exec -it mongodb mongosh fintrack
```

```
db.activity_events.countDocuments({})
db.activity_events.find().sort({occurredAt:-1}).limit(5).pretty()
db.activity_events.getIndexes()
db.report_snapshots.find().pretty()
```

```
indexes: _id_, user_time, user_type_time, event_id_unique
```

`event_id_unique` is the idempotency guard for the activity writer. It cannot use the Postgres `processed_events` ledger, because its write and that ledger live in different databases and cannot share a transaction — claiming first and then failing to write would lose the event while the ledger insisted it was handled.

Redis

```
docker compose exec -it redis redis-cli
```

```
KEYS analytics:*           # cached analytics, one set per user
TTL analytics:user:<id>:portfolio
KEYS bull:*                 # BullMQ queue state
LLEN bull:analytics:wait    # queue depth – growing means workers cannot keep up
```

8. Tests, benchmarks and load results

[contents ↑](#)

The test suite

```
docker compose up -d
docker compose --profile app stop api worker nginx # if the full stack is running
npm test
```

```
Test Suites: 11 passed, 11 total
Tests:       153 passed, 153 total
Time:        ~50 s
```

These run against **real** PostgreSQL, MongoDB and Redis. Mocks cannot demonstrate row locking, transaction rollback or cache fallback — the behaviour only exists against the real thing. They run serially because several suites truncate a shared database.

A few of the things they assert:

Test	Why it matters
A forced failure mid-transaction rolls back the balance <i>and</i> the outbox row	Proven with a real <code>NUMERIC</code> overflow, not a mock
Two concurrent withdrawals: exactly one succeeds	The lost-update race, run for real
Five simultaneous retries of one key → one transaction	Idempotency under contention, not just sequentially
Analytics correct with Redis genuinely stopped	The container is stopped, not a client mocked to throw
Three trades applied in reverse order → correct state	Regression guard for the ADR-019 bug

Index benchmark

```
npm run seed -- --users=200 --transactions=500000 # ~30 s
docker compose exec -T postgres psql -U fintrack -d fintrack -f db/benchmark/explain.sql
```

Transaction history query · 500,018 rows

with index	 0.153 ms	Index Scan · 23 buffers · no sort
without it		38.078 ms
Bitmap Heap Scan → top-N heapsort · 2,246 buffers · reads 2,665 rows to return 20		
≈ 249× faster, ≈ 98× fewer buffers touched		

The index is dropped inside a transaction that is rolled back, so a failed run cannot leave it missing.

The interesting part is that the slow plan is **not** a sequential scan. Postgres falls back to the idempotency index, which also starts with `account_id`, finds the 2,665 matching rows, then has to read every one from the heap and sort them to return 20. The cost is in reading the rows, not in finding them.

Load tests

```
docker compose --profile app up -d --scale api=3 --scale worker=2
k6 run load/smoke.js # is the stack wired up?
```

```
k6 run load/read-heavy.js      # cached analytics
k6 run load/write-heavy.js     # deposits under lock contention
```

Scenario	Throughput	p50	p95	Failed
Read-heavy (cached analytics, 20 VUs)	5,107 req/s	2.72 ms	6.46 ms	0.000%
Write-heavy (deposits, one account, 10 VUs)	361 req/s	9.09 ms	94.22 ms	0.000%

The write run produced **16,437 outbox rows**, which two workers drained to a backlog of **zero**.

What these numbers are not. The load generator shares four CPU cores with the entire stack, so they are a floor rather than a ceiling. The read test runs at a 100% cache hit rate against a single user — it measures the cached path, which is the point of the cache and also the limit of the number. 500k rows fit comfortably in page cache at this VM size; behaviour at 50M rows is untested. None of it is a capacity claim.

Recording those caveats beside the figures is the difference between a measurement and a marketing number.

9. Scaling it and proving it scaled

[contents ↑](#)

```
docker compose --profile app up -d --scale api=3 --scale worker=2
```

Prove requests actually distribute

```
for i in $(seq 1 30); do
  curl -s -D - -o /dev/null http://localhost:8080/health | grep -i '^x-instance-id'
done | sort | uniq -c
```

```
10 x-instance-id: api-bfa969b0a202
10 x-instance-id: api-d8b2ad8002a8
10 x-instance-id: api-ebcbc4688633
```

Each container reports its own hostname. An even split across three instances.

Prove the API is stateless

A token issued by one instance is accepted by all of them — nothing session-related is held in process memory, which is what lets any instance serve any request:

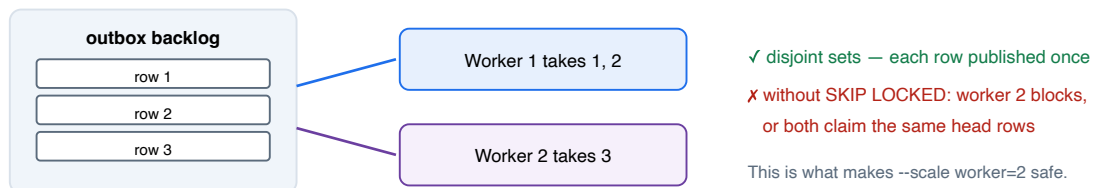
```
HTTP/1.1 200 OK x-instance-id: api-bfa969b0a202
HTTP/1.1 200 OK x-instance-id: api-d8b2ad8002a8
HTTP/1.1 200 OK x-instance-id: api-ebcbc4688633
...
```

Prove two workers do not double-process

```
docker compose exec -T postgres psql -U fintrack -d fintrack -c \
  "SELECT count(*) AS outbox FROM outbox_events;
  SELECT processor, count(*) FROM processed_events GROUP BY 1;"
```

Each event appears once per processor, never twice. The publisher claims rows with `FOR UPDATE SKIP LOCKED`, so two publishers take **disjoint** sets rather than both grabbing the head of the backlog.

Why SKIP LOCKED matters with two publishers



Scale the workers alone when the queue backs up but request latency is fine — that independence is the concrete reason the API and worker are separate processes:

```
docker compose --profile app up -d --scale worker=4
```

10. Troubleshooting

[contents ↑](#)

Symptom	Cause	Fix
<code>JWT_SECRET</code> must be at least 32 characters	<code>.env</code> missing or unset	<code>openssl rand -hex 32</code> , paste into <code>.env</code>
<code>@prisma/client</code> did not initialize	Client not generated	<code>npx prisma generate</code>
<code>allow-scripts</code> warning, Prisma fails	npm 11 blocks install scripts	<code>npm approve-scripts prisma @prisma/engines @prisma/client</code>
<code>docker: unknown command: compose</code>	Homebrew compose not linked as a plugin	see the symlink in section 3
Cannot connect to the Docker daemon	Colima not running	<code>colima start --cpu 4 --memory 4 --disk 40</code>
Analytics stay empty	The worker is not running	<code>npm run dev:worker</code>
Analytics stale by a few seconds	Normal — eventually consistent by design	wait, or read the worker log
Tests fail with odd queue counts	A containerised worker is eating the jobs	<code>docker compose --profile app stop api worker nginx</code>
<code>unpublished</code> count keeps growing	Worker down, or Redis unreachable	check the worker log and <code>docker compose ps</code>
Port 4000 or 8080 already in use	An earlier run is still alive	<code>lsof -ti:4000 xargs kill</code>
Cost basis looks wrong (2500 / 750)	Pre-ADR-019 build	<code>git pull</code> — fixed in <code>8b7004a</code>

Start over with clean data

```
docker compose down -v      # -v deletes the volumes and all data
docker compose up -d
npx prisma migrate deploy
```

Read the logs

```
docker compose logs -f worker
docker compose logs -f api
docker compose logs -f nginx
```

Shut everything down

```
docker compose down          # keeps data in volumes
docker compose --profile app down # also stops api/worker/nginx
docker compose down -v      # deletes data too
colima stop                  # stop the VM entirely
```

11. The 90-second demo

[contents ↑](#)

If you have to show this quickly — in an interview, or to a reviewer.

```
docker compose --profile app up -d --scale api=3 --scale worker=2
API=http://localhost:8080
TOKEN=$(curl -s -X POST $API/auth/register -H 'Content-Type: application/json' \
  -d '{"email":"demo-$(date +%s)'@x.dev',"name":"Demo","password":"correct-horse-battery"}' | jq -r .token)
ACC=$(curl -s -X POST $API/accounts -H "Authorization: Bearer $TOKEN" \
  -H 'Content-Type: application/json' -d '{"accountType":"BROKERAGE"}' | jq -r .account.id)
```

#	Show	Say
1	<code>curl -s \$API/health jq</code>	Three subsystems, probed independently. Postgres down is a 503; Redis down is not.
2	Deposit, buy twice, sell once	Each is one database transaction: row lock, insert, balance, outbox — all or nothing.
3	<code>/analytics/portfolio</code>	Updated by a worker, not by the request. The API never waited for it.
4	Run it twice, show <code>x-cache</code>	MISS then HIT. Cache-aside, 10-minute TTL, invalidated by the worker on write.
5	<code>docker compose stop redis</code> , run again	Still 200, still correct. The cache is an optimisation; Postgres is the truth.
6	<code>/activity</code>	Same events from MongoDB, with per-type metadata a relational table would store as NULLs.
7	The 30-request instance-id loop	Three instances sharing load; a token from one works on all — nothing in process memory.

Then say the thing that actually matters:

“The API never waits for the worker, and the worker can be restarted, duplicated or crashed without losing an event or double-counting one.”

Questions you should be ready for

Question	Where the answer lives
Why PostgreSQL and MongoDB?	§5.4 — heterogeneous event metadata, no joins, no transactions needed
What happens if Redis dies?	§6.3 — degrades to Postgres, still 200
How do you avoid double-processing?	§6.5 — ledger in Postgres, unique index in Mongo, and why they differ
What if the same POST arrives twice?	§6.1 — two mechanisms, sequential and concurrent
Why is the outbox worth the complexity?	§6.4 — the dual-write problem, drawn
What breaks at 100× traffic?	§9 — API and worker scale independently; write contention is the real ceiling
Prove the index helps	§8 — 0.153 ms vs 38.078 ms, with the plans

FinTrack — github.com/kanhaiyak23/FinTrack

Companion documents in the repository: `docs/RUNBOOK.md` (the text version of this guide), `docs/architecture.md` (design and failure modes), `docs/decisions.md` (19 decisions with their rejected alternatives), `postman/` (26 chained requests).

Every command in this document was executed against the codebase before being written down. Three did not survive that and were corrected rather than published.